EE5904/ME5404 Part II

Project 2: Q-Learning for World Grid Navigation

Hridik Kajaria

A0263828U

E1011186@u.nus.edu

INTRODUCTION

This report explains the implementation of Q-learning algorithm with the ε-greedy exploration method to navigate a robot in a 10x10 world grid. It considers different discount factors and exploration probabilities to understand the principles of and issues related to the algorithm.

A robot is placed at the top left grid box, at start position 1, in a 10x10 grid map. The grids are numbered in succession from top to bottom in a column with the goal position at 100 at the bottom right corner. *Figure 1 10x10 world grid* below shows the world grid for robot navigation.

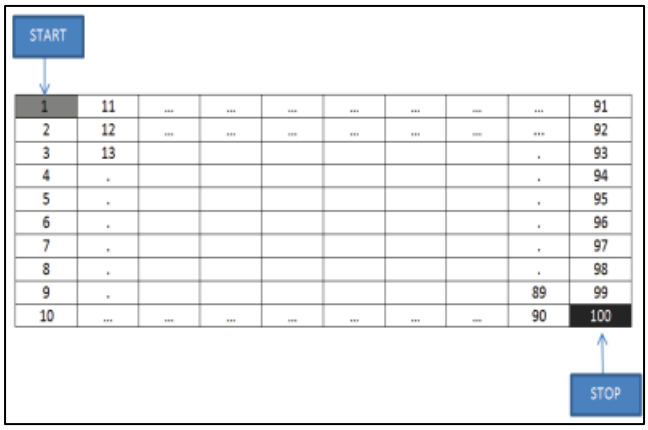


Figure 1 10x10 world grid

The Q-learning algorithm is based on the principle of maximizing rewards while traversing the grid. A reward function is supplied that contains a particular value for each action possible at a particular state. The number of actions possible in the current case are 4: 1 → up, 2 → right, 3 → down, 4 → left. The reward function is a 100x4 array with values for each action possible at each state. The action selection at any state is determined through the ε-greedy exploration algorithm, with two different values of discount factors and 4 different exploration probability functions. A comparison is drawn between different settings for better understanding of the dependence on the underlying parameters.

The Q-learning algorithm is run 10 times with particular settings and averaged to obtain a general result. Each run is associated with a maximum number of 3000 trials.

IMPLEMENTATION

The implementation procedure is shown in *Figure 2 Q-learning algorithm implementation procedure.*

- The total number of runs (10) and maximum trials (3000) are specified. A count is initialized that increments every time the robot reaches state 100. An execution time empty list is initialized to record the execution time for each run.
- For each run, the time starts recording and the parameters are initialized. A 100x4 Q matrix is initialized with zero values. The error threshold is kept at 0.005 and a trial is initiated. For each

trial within the limit, parameters are initialized and a time step is initiated. The time step terminates if the robot reaches the goal position or the learning rate is greater than the error threshold or the time step reaches 200. The trial terminates if the Q-function converges to optimal values.

- The best action for each time step is determined by the ε-greedy algorithm which comprises exploration and exploitation. Exploration refers to randomly selecting an action and weighing its probability. Exploitation refers to selecting the highest reward action form the Q values.
- The average time consumed over 10 runs for all goal-reaching runs is calculated and the optimal policy obtained is determined from the Q-values.

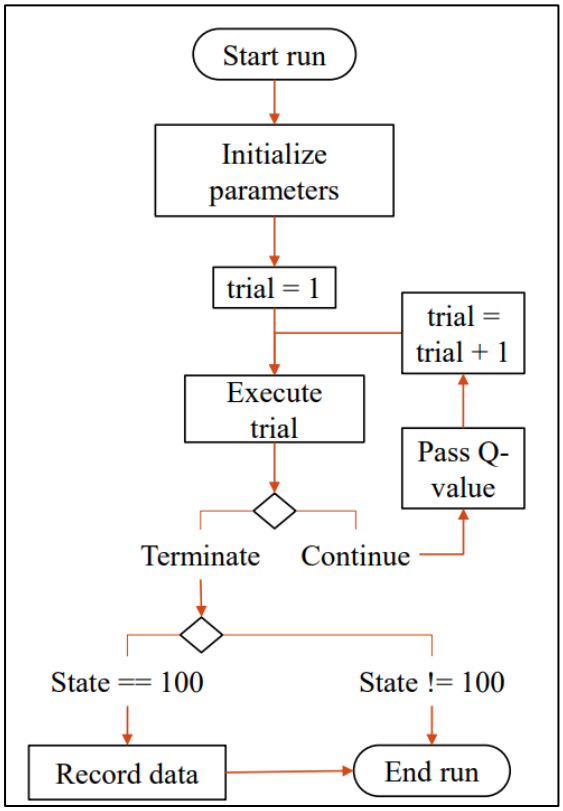


Figure 2 Q-learning algorithm implementation procedure

RESULTS AND DISCUSSION

Table 1 Parameter values and performance for Q-learning

S. No.	ε_k, α_k	No. of goal-reached runs		Execution time (sec)	
		$\gamma = 0.5$	$\gamma = 0.9$	$\gamma = 0.5$	$\gamma = 0.9$
1	1/k	0	0	NaN	NaN
2	100/(100+k)	10*	10	0.5611	2.0136
3	(1+log(k))/k	0	0	NaN	NaN
4	(1+5log(k))/k	10*	9	1.0674	4.3966

The above table shows the effect of discount factor and exploration probability function on the optimal policy for navigation.

Key takeaways: -

- The robot reaches the goal position for only two out of four functions. The first two functions are comparable to each other. The decrease rate for function 1 is higher than that of function 2. This causes the exploration ability to reduce in the case of the former. To illustrate, at time step

100, function 1 probability is 1/100 = 0.01 and function 2 probability is 100/200 = 0.5. The exploration strategy is dependent upon the probability, i.e., if a random number is less than ε_k, then a random action may be selected. At time step 100, function 2 gives a greater margin for exploration to occur, as compared to function 1. Similarly, for functions 3 and 4, the decreasing rate of function 4 is slower than that of function 3, and the goal is reached. The difference in the functions is shown in *Figure 3 Plots of different functions used* below.

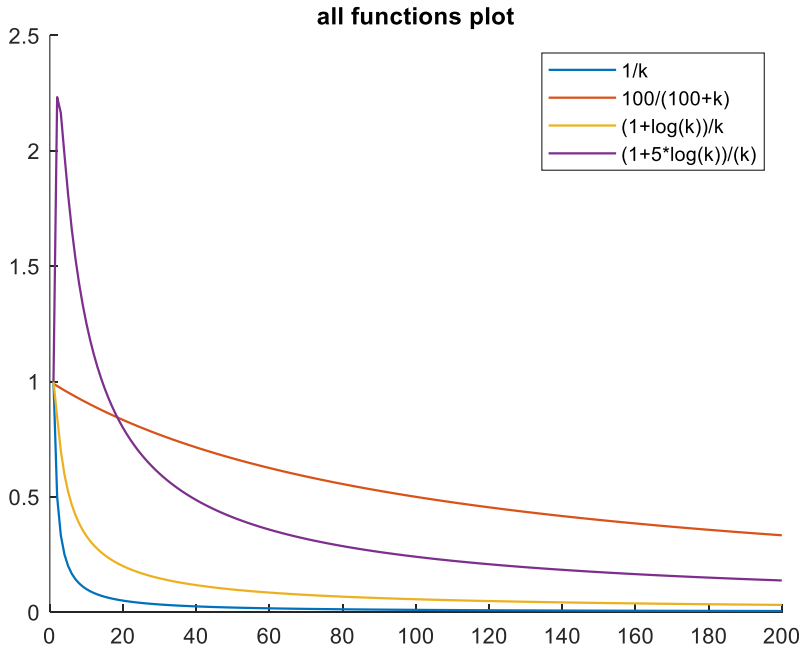


Figure 3 Plots of different functions used

- The discount factor γ has a significant impact on the execution time of all goal reaching runs. A higher discount factor means that the robot is allowed to consider long-term benefits and gives higher weight to future rewards. This causes the robot to take more time for execution, as is depicted in *Table 1* for functions 2 and 4.
- Upon comparing functions 2 and 4, it is observed that in the case of the former, the goal is reached for both the discount factors, while for the latter, only 9 times out of 10, the goal is reached when discount factor is 0.9. This can be related to the combination of discount factor and decreasing rate of the functions. The number of trials is also shown in Figure 4.
- The discount factor 0.5 makes the robot reach the goal position but it is a highly iterative process and does not produce the optimal path. Whereas, if the discount factor is 0.9, the robot reaches the goal position in just 18 steps. The asterisk (*) in *Table 1* represents the parameters that do not produce an optimal policy for the robot to complete navigation.

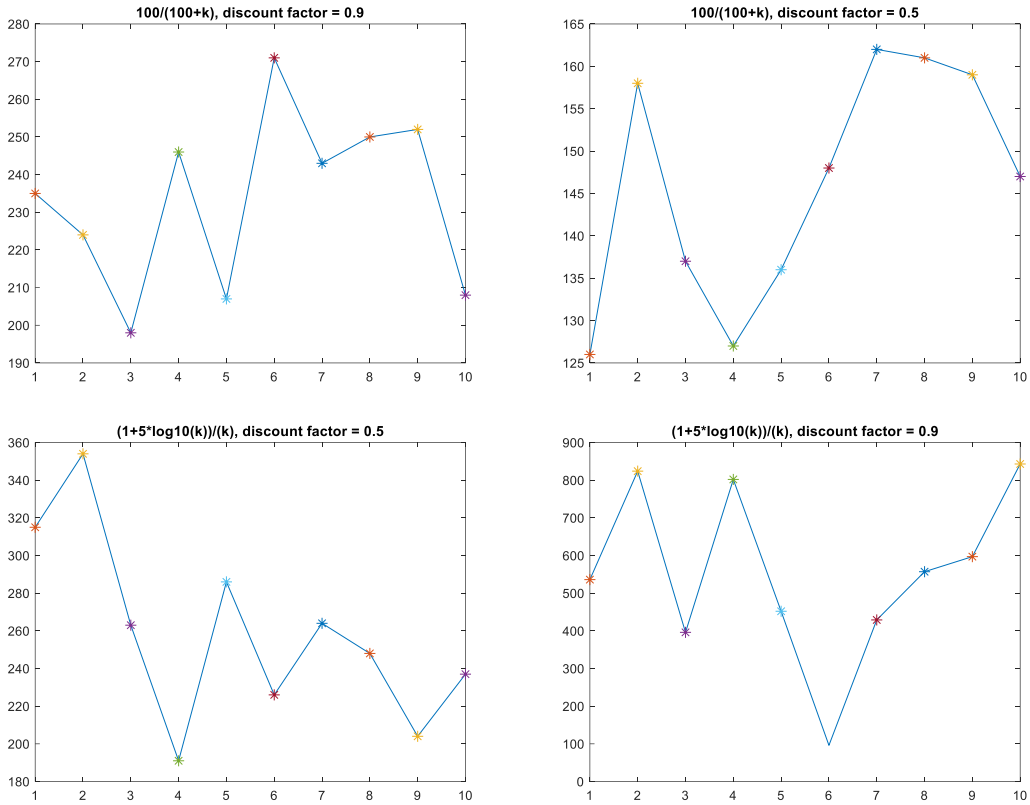


*Figure 4 Number of trials for goal reaching runs represented by * for the four different cases. Bottom right plot shows 9 counts of reaching the goal.*

<u>Optimal policy</u>

1. Column vector:
   $[3, 2, 3, 3, 3, 3, 3, 2, 2, 2, 2, 2, 2, 2, 2, 3, 3, 1]^T$
   The last element is the final goal position (state 100)
2. Figure 5 below shows the optimal actions recorded at each state. The triangles point towards the best action to be taken.

Figure 5 Optimal actions at each state defining the optimal policy.

3. Shows the optimal path taken by the robot to reach state 100. The robot traverses just 18 grids to reach the goal. The total reward is equal to 10932 points. 'x' marks the spot (of goal position).

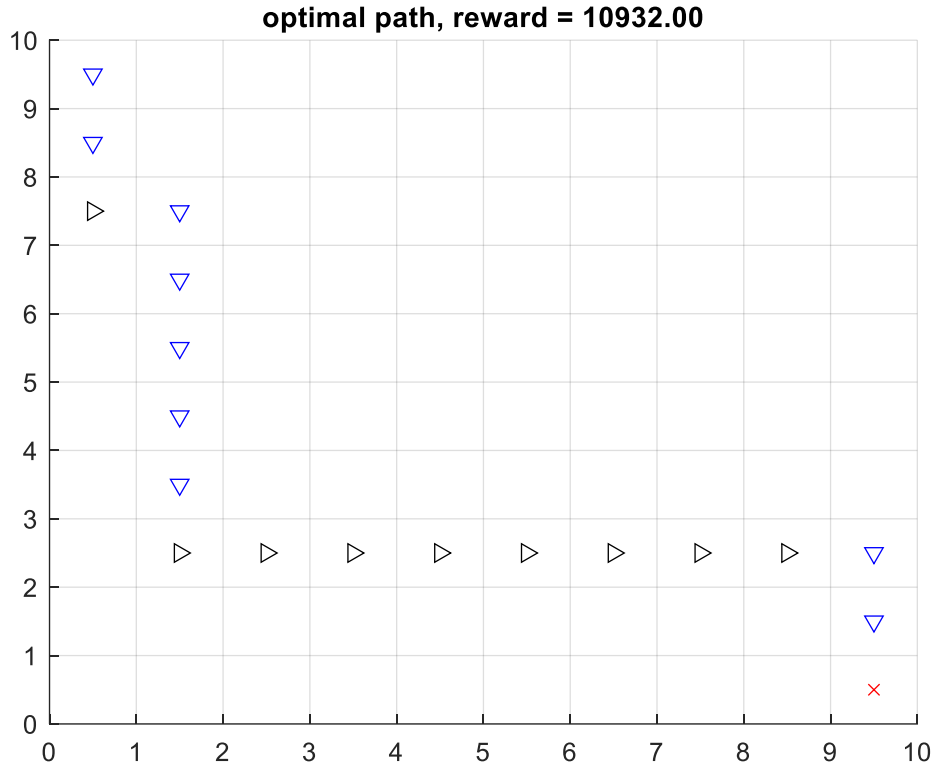


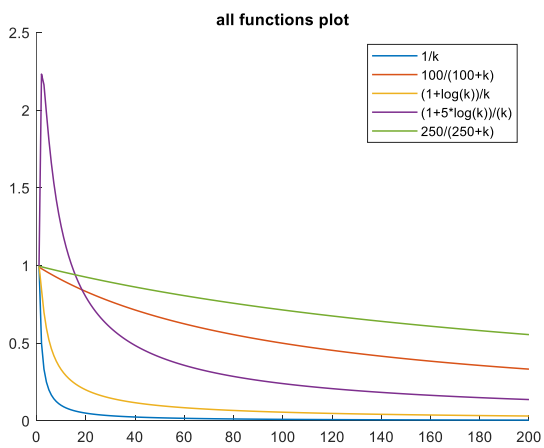
Figure 6 Optimal path taken by the robot.

TASK 2

Using *Table 1* Parameter values and performance for Q-learninga compromise can be struck for discount factor and probability function for Q-learning.

The function must have a slow decreasing rate and the discount factor must allow the robot to look to the future.

Parameters for task 2:

$\gamma = 0.95$

$\varepsilon_k = 50/(50+k)$ *(green plot in figure below)*